```
      raise ValueError("Invalid user")
    if self._since is None:
      self.since = datetime.datetime.now()
```

You can observe that the *User* is not frozen as we want it to be mutable. However, we do not want all the properties mutable. The identity field like *_name* and fields like *_since* are not expected to be modified. Then, how can this be controlled?

Python 3 offers the so-called *Descriptor* protocol. It helps us in defining the getters and setters appropriately. Let us add the getters to all the three fields of *User* using the *@property* decorator:

```
@property
def name(self) -> Name:
  return self._name

@property
def phone(self) -> PhoneNumber:
  return self._phone

@property
def since(self) -> datetime.datetime:
  return self._since
```

And the setter for the phone field can be added using *@<field>.setter* decoration:

```
@phone.setter
def phone(self, phone: PhoneNumber) -> None:
  if phone is None:
    raise ValueError("Invalid phone")
  self._phone = phone
```

The *User* can also be given a method for easy print representation by overriding the *__str__()* function:

```
def __str__(self):
  return self.name.value + " [" + str(self.phone.value) + "]
since " + str(self.since)
```

With this the entities and the value objects of the domain model are ready. Creating an exception class is as easy as follows:

```
class UserNotFoundException(Exception):
  pass
```

The only other one remaining in the domain model is the *UserRepository*. Python offers a useful module called *abc* for building abstract methods and abstract classes.

Since *UserRepository* is only an interface, we can use the *abc* package.

Any Python class that extends *abc.ABC* becomes abstract. Any function with the *@abc.abstractmethod* decorator becomes an abstract function. Here is the resultant structure of *UserRepository*:

```
from abc import ABC, abstractmethod

class UserRepository(ABC):
  @abstractmethod
  def fetch(self, name:Name) -> User:
    pass
```

The *UserRepository* follows the repository pattern. In other words, it offers appropriate CRUD operations on the *User* entity without exposing the underlying data storage semantics. In our case, we need only *fetch()* operation since *FindService* only finds the users.

Since the *UserRepository* is an abstract class, we cannot create instance objects from this class. A concrete class must implement this abstract class for object creation.

The data layer

The *UserRepositoryImpl* offers the concrete implementation of *UserRepository*:

```
class UserRepositoryImpl(UserRepository):
  def fetch(self, name:Name) -> User: pass
```

Since the *AddService* stores the data of the users in a MySQL database server, the *UserRepositoryImpl* also must connect to the same database server to retrieve the data. The code for connecting to the database is given below. Observe that we are using the connector library of MySQL.

```
from mysql.connector import connect, Error

class UserRepositoryImpl(UserRepository):
  def fetch(self, name:Name) -> User:
    try:
      with connect(
        host="mysqldb",
        user="root",
        password="admin",
        database="glarimy",
      ) as connection:
        with connection.cursor() as cursor:
          cursor.execute("SELECT * FROM ums_users where
name=%s", (name.value,))
          row = cursor.fetchone()
          if cursor.rowcount == -1:
```